

CA-PQ Whitepaper

A Hybrid Post-Quantum KEM Framework Built From Zero

CA-PQ Project

27 September 2026

Companion to the Yellow Paper (formal specification) and the Red-Team Report

Abstract

CA-PQ is a post-quantum cryptography framework that refuses the usual trade-offs. Instead of choosing between battle-tested classical ECDH and larger post-quantum KEMs, it combines both—plus a deterministic chaotic-stream layer—under a single transcript, in a single dependency-free Rust tree where every primitive is hand-written and every claim is tested. Two PQ profiles (ML-KEM-768 at NIST level 3, ML-KEM-512 at level 1) feed two hybrid combiners alongside a hand-written X25519 leg. The result, on commodity hardware: ~17k PQ encapsulations per second, ~2.7k hybrid pairs per second, 828–1 148-byte ciphertexts, zero third-party code, and a red-team file documenting exactly what was attacked, what broke, and what remains open.

Contents

1	The problem	2
2	Design principles	2
3	Architecture	3
4	The chaotic layer: what it is and is not	3
5	Security analysis	3
6	Performance	4
7	Positioning	5
8	Limitations and roadmap	5
9	Documents	5

1 The problem

Three forces collide in real-world cryptography migration:

1. **Quantum horizon.** X25519/ECDH falls to a cryptographically relevant quantum computer. Lattice KEMs (ML-KEM/FIPS 203) are the standardized answer, but migration takes years and “pure PQ” deployments lose the classical track record.
2. **Hybrid demand.** The conservative answer—combine classical and PQ so the session holds if *either* holds—is now best practice in TLS experimentation. But hybrids double implementation surface and dependency load.
3. **Supply-chain reality.** Each dependency is trusted code. PQC libraries pull in big-integer crates, RNGs, vendor SIMD intrinsics, and build scripts. Auditing the union is impractical for small teams.

CA-PQ’s answer: a hybrid framework with **zero dependencies**, **deterministic APIs** (no RNG to subvert), and **tested honesty** (every security claim paired with the test that would falsify it).

2 Design principles

Hybrid by construction, not by glue.

The combiner is a first-class primitive ($ss = \text{Hash}(ss_x || ss_{pq} || \text{transcript})$), not two KEMs taped together. Per-leg failure semantics are specified and tested (X fails closed, PQ implicitly rejects).

Zero dependencies, verifiably.

`Cargo.lock` contains only `chaos-*` crates. Hash, sponge, curve, lattice math, and wiping are hand-written from public specifications.

Determinism as a security feature.

`keygen(seed)`, `encaps(pk, coins, nonce)`—all randomness is caller-supplied. No RNG means no RNG failures, no hidden state, and byte-exact test vectors for every operation.

No floats in the key path.

The chaotic engine runs in Q32.32 fixed point: cross-platform bit-identical, no FPU timing or rounding variance.

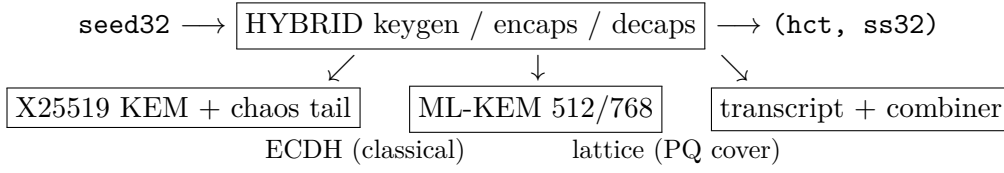
Performance without compromise of auditability.

Every optimization (AVX2/NEON/SSE2 tiers, Barrett reductions, batch APIs) is gated behind differential tests plus full-length KATs, and every “clever” change that measured ~ 0 was reverted on record.

Honesty as architecture.

The threat model lists non-claims with the same care as claims. The red-team file records defects found, including the methodology traps that produced them.

3 Architecture



X leg (`chaos-kem`, sysid PQ04). Hand-written X25519 ECDH (5×51 -bit limbs, Montgomery ladder, addition-chain inverse) wrapped with a chaotic-stream tail: $\text{tail} = \text{keystream}(\text{expand}(\text{XOF}(\text{shared} \parallel \text{transcript})))$. $ss_x = \text{Hash}(\text{tail} \parallel \text{shared} \parallel \text{transcript})$. Contributory zero-checks on both sides.

PQ leg (`chaos-mlkem`). ML-KEM-768 and ML-KEM-512 from one generic core—NTT with compile-time twiddle tables, SHAKE-based sampling, CBD noise, Fujisaki–Okamoto transform with implicit rejection, batched matrix sampling, batch encaps API.

Combiner (`chaos-hybrid`, sysids PQH2/PQH1). Subseed derivation, joint transcript commit $\parallel ct_x \parallel ct_{pq} \parallel \text{nonce}$, $ss = \text{Hash}(ss_x \parallel ss_{pq} \parallel \text{transcript})$. Secure if either leg holds.

DEM (`chaos-aead`). XOR stream + 16-byte keyed MAC, heapless API, constant-time verify.

4 The chaotic layer: what it is and is not

The engine integrates a damped 4D quadratic flow (Lorenz-28 regime: $a = 10$, $b = 8/3$, $c = -1$, $d = 28$) in Q32.32 fixed point with RK4 at $dt = 0.001$, discarding $N_{\text{TRANSIENT}} = 1000$ steps and sampling every 10th step through a bit-folding extractor. Measured largest Lyapunov exponent ≈ 0.6 – 0.9 (positive: chaotic, single—*chaotic*, *not hyperchaotic*, stated honestly after re-parameterization from a collapsed regime the tests caught).

What it **is**: a fast local stream expander and KDF with 256-bit seeds, strong seed sensitivity (independent seeds: Hamming 0.500 flat from 50 steps), and validated whiteness (NIST-style 8/8, `ent` 7.99999, 200 KB soak).

What it **is not**: asymmetry. Red-team gates prove the negative—raw drive is a slow predictable signal (lag-1 $r \approx 0.97$ at sync sampling) and every observer synchronizes to every receiver on the same drive. Drive is never published, never trusted. Worst case (§5) reduces to the asymmetric roots.

Known nuance: rare materials capture bit-exactly by step $\sim 2000+$ (periodic-window dynamics). Production tails end at step 1640, related materials are unreachable (XOF), tails are unobservable (hashes only)—INFO-level, sensor-pinned, fully analyzed in the Red-Team Report.

5 Security analysis

Per-leg foundations. X25519: RFC 7748 shape pinned to independent big-int goldens (clamping, masking, ladder constant, inversion chain, encoding). ML-KEM: FIPS-203 shape pinned to full-length KAT agreement with an independent transcription (both profiles; no interop claim). BLAKE3/SHAKE: official vectors + split-invariance + rate-boundary oracles.

Combiner argument. The transcript binds commit, both ciphertexts, and nonce; *ss* binds both leg secrets plus transcript. Tampering X fails closed (**None**); tampering PQ yields unrelated *ss* (implicit rejection, deterministic); wrong keys fail; zero peer points fail closed on every combiner (subgroup confinement: clamped scalars $\times$ torsion = 0, gated both sides—tested). Cross-profile confusion is impossible by type (distinct sizes) and separated by sysid + hash domains (verified by full-stack KATs in both profiles).

KEM hygiene. FO transform with constant-time select; re-encryption is unconditional (timing smoke: valid:tampered $\approx$ 1:0.93—no early-exit oracle). KEM MAC is keyed, verified before use; AEAD verifies before decrypting (constant-time compare).

Misuse model (documented, tested). Nonce reuse degrades to stream-XOR properties (standard, not resisted). Caller RNG failures produce weak-but-functional keys (deterministic API contract). ML-KEM dk arrays need `burn_dk` (plain arrays can’t self-wipe; in-tree flows do).

What is explicitly not claimed. No chaos asymmetry; no NP-hardness/hidden-subgroup claims; 256-bit seeds targeting $\sim$ 128-bit classical security; not constant-time beyond branchless swaps and tag comparison (bit-level timing accepted, no secret-dependent exits/lengths—verified); no external audit (the ask of this paper’s readers).

6 Performance

Measured on Intel i5-14400F via `ca-pq bench` (shared runner—expect $\pm$ 10–20% run variance). Throughputs in operations per second unless noted.

Table 1: Portable build (default x86-64: SSE2 BLAKE3 tier, scalar Keccak/NTT).

Operation	Throughput
BLAKE3 1 MiB	$\sim$ 1 500 MiB/s
BLAKE3-keyed 64 B	$\sim$ 11 M ops/s
SHAKE128 bulk	$\sim$ 260 MiB/s
Keystream (chaotic)	$\sim$ 3.3 MiB/s
X25519-KEM encaps / decaps	$\sim$ 7.4k / $\sim$ 11k
ML-KEM-768 encaps / decaps	$\sim$ 14.4k / $\sim$ 14.1k
ML-KEM-768 batch-8 (per element)	$\sim$ 18.3k
ML-KEM-512 encaps	$\sim$ 21.7k
Hybrid-768 pair	$\sim$ 2.6k pairs/s
Hybrid-512 pair	$\sim$ 3.0k pairs/s

Cumulative AVX2-vs-original-portable gains on the PQ path: encaps +58%, decaps +100%, pair +54%. The X25519 leg is SIMD-untouched by design (identical both builds).

AEAD tag: 16 bytes. Release CLI binary: $\sim$ 631 KiB portable / $\sim$ 683 KiB AVX2 (LTO, stripped). Tree: $\sim$ 8 200 lines of Rust, zero dependencies.

Table 2: AVX2 build (RUSTFLAGS="-C target-feature=+avx2").

Operation	Throughput
BLAKE3 1 MiB	~2 000–2 500 MiB/s
ML-KEM-768 encaps / decaps	~17.1k / ~18.7k
ML-KEM-768 batch-8 (per element)	~21.6k
ML-KEM-512 encaps	~25.4k
Hybrid-768 pair	~2.7k pairs/s

Table 3: Wire sizes in bytes (fixed by type).

Object	X-KEM	ML-KEM-768	ML-KEM-512	Hybrid-768	Hybrid-512
Public key	68	1 184	800	1 284	900
Secret key (stored)	64 (32 seed)	2 400	1 632	2 464 (32 seed)	1 696 (32 seed)
Ciphertext	60	1 088	768	1 148	828
Shared secret	32	32	32	32	32

7 Positioning

CA-PQ does not compete with broad PQ libraries on breadth (one lattice family, two profiles) or with audited TLS stacks on deployment readiness. Its position:

Dimension	CA-PQ	Typical PQ library
Dependencies	0 (verified in lockfile)	OS-scale trees
PQ + classical	hybrid by construction	usually one or glued
Deterministic API	yes (caller RNG)	usually OS RNG inside
Float-free key path	yes (Q32.32)	rare
External audit	no (the ask)	some
KAT depth	full-length, multi-implementation	official vectors

Within the tree, profiles trade size for speed honestly: 512 is $\sim 1.6\times$ faster with $\sim 30\%$ smaller wires at NIST level 1; 768 holds level 3; hybrids add $\sim 60/828\text{--}1\,148$ B over the PQ leg for classical + stream robustness.

8 Limitations and roadmap

In order: **external audit** (the blocker for any production use), transient-margin maintenance, NEON/SSE2 Keccak–NTT completion detail (SSE2 NTT explicitly rejected with numbers; NEON done), Kani proofs, TestU01/DIEHARDER campaign. Non-goals: new primitives, protocol design beyond KEM+DEM, masking/countermeasure engineering (documented stance).

9 Documents

Yellow Paper (formal specification); Red-Team Report (audit R1–R6, A1–A4); Threat Model (claims, non-claims, gates); SPEC (frozen parameters, normative, code-adjacent); PLAN (build history v1–v23 with measurements); CHANGELOG (v2.0.0 current).